Name: Shryitha Suddala

Hall Ticket: 2303A51637

Batch: 29

Assessment: 12.5

Task 1: Merge Sort Implementation

Prompt:

Write a Python program that implements the Merge Sort algorithm

Code:

```python
# Write a Python program that implements the Merge Sort algorithm
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    merged = []
    i, j = 0, 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged

# Example usage:
if __name__ == "__main__":
    arr = [64, 34, 25, 12, 22, 11, 90]
    print("Original array:", arr)
    sorted_arr = merge_sort(arr)
    print("Sorted array:", sorted_arr)
```

Output:

```
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]
```

Significance:

Merge Sort is an efficient **divide-and-conquer sorting algorithm** that divides the array into smaller parts and merges them after sorting. It guarantees a **time complexity of O(n log n)** in all cases, making it suitable for large datasets. This algorithm is also **stable**, meaning the relative order of equal elements is preserved.

Task 2: Binary Search with AI Optimization

Prompt:

create a binary search function that finds a target element in a sorted list

Code:

```python
# create a binary search function that finds a target element in a sorted list
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
# Example usage:
if __name__ == "__main__":
    arr = [11, 22, 34, 64, 90]
    target = 34
    result = binary_search(arr, target)
    if result != -1:
        print(f"Element {target} found at index {result}")
    else:
        print(f"Element {target} not found in the array")
```

Output:

```
Element 34 found at index 2
```

Significance:

Binary Search efficiently finds an element in a **sorted list** by repeatedly dividing the search space into halves. It significantly reduces the number of comparisons compared to linear search. With a **time complexity of O(log n)**, it is ideal for fast searching in large datasets.

Task 3: Smart Healthcare Appointment Scheduling System

Prompt:

suggest a suitable searching and sorting algorithms for Healthcare Appointment Scheduling System that stores appointment records containing appointment ID,patient name, doctor name, appointment time, and consultation fee.and justify the choice of algorithms.

Code:

```
# suggest a suitable searching and sorting algorithms for Healthcare Appointment Scheduling System that stores appointment records containing appointment ID,
# patient name, doctor name, appointment time, and consultation fee.and justify the choice of algorithms.
from heapq import merge
#For a Healthcare Appointment Scheduling System, I would suggest using the Merge Sort algorithm for sorting the appointment records and the Binary Search algorithm for searching through the sorted records.
#Justification for Merge Sort:
"""1. Merge Sort is a stable sorting algorithm, which means it preserves the relative order of records with equal keys (e.g., appointment time).
This is important in a scheduling system where the order of appointments may matter.
2. Merge Sort has a time complexity of O(n log n) in all cases, making it efficient for sorting large datasets,
 which is likely in a healthcare system with many appointments."""
#Justification for Binary Search:
"""1. Binary Search is efficient for searching through sorted data, with a time complexity of O(log n).
 This makes it suitable for quickly finding specific appointment records based on criteria such as appointment ID or patient name.
2. Since the appointment records will be sorted using Merge Sort, Binary Search can be effectively utilized to retrieve information
without having to scan through the entire list, thus improving performance in a real-time scheduling system.
Overall, the combination of Merge Sort for sorting and Binary Search for searching provides an efficient and effective way to manage appointment records
 in a Healthcare Appointment Scheduling System, ensuring that the system can handle large volumes of data while maintaining quick access to specific records when needed."""
# Example implementation of Merge Sort and Binary Search for the Healthcare Appointment Scheduling System
"""class Appointment:
    def __init__(self, appointment_id, patient_name, doctor_name, appointment_time, consultation_fee):
        self.appointment_id = appointment_id
        self.patient_name = patient_name
        self.doctor_name = doctor_name
        self.appointment_time = appointment_time
        self.consultation_fee = consultation_fee
def merge_sort(appointments):
    if len(appointments) <= 1:
        return appointments
    mid = len(appointments) // 2
    left = merge_sort(appointments[:mid])
    right = merge_sort(appointments[mid:])
    return merge(left, right)
def merge(left, right):
    merged = []
    i, j = 0, 0
    while i < len(left) and j < len(right):
        if left[i].appointment_time < right[j].appointment_time:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged
def binary_search(appointments, target_id):
    low = 0
    high = len(appointments) - 1
    while low <= high:
        mid = (low + high) // 2
        if appointments[mid].appointment_id == target_id:
            return appointments[mid]
        elif appointments[mid].appointment_id < target_id:
            low = mid + 1
        else:
            high = mid - 1
    return None
# Example usage:
if __name__ == "__main__":
    appointments = [
        Appointment(1, "John Doe", "Dr. Smith", "2024-07-01 10:00", 100),
        Appointment(2, "Jane Doe", "Dr. Brown", "2024-07-01 11:00", 150),
        Appointment(3, "Alice Johnson", "Dr. Smith", "2024-07-01 09:00", 120),
    ]
    sorted_appointments = merge_sort(appointments)
    target_id = 2
    result = binary_search(sorted_appointments, target_id)
    if result:
        print(f"Appointment found: {result.patient_name} with {result.doctor_name} at {result.appointment_time}")
    else:
        print("Appointment not found.")"""
```

Output:

```
Appointment found: Jane Doe with Dr. Brown at 2024-07-01 11:00
```

Significance:

Using **Merge Sort** allows appointment records to be organized efficiently based on appointment time or consultation fee. **Binary Search** enables quick retrieval of

appointment details using appointment ID. This combination improves system performance and ensures efficient management of large numbers of appointments.

Task 4: Railway Ticket Reservation System

Prompt:

suggest a suitable searching and sorting algorithms for Railway Ticket Reservation System that stores ticket ID, passenger name, train number, seat number, and travel date.and justify the choice of algorithms.

Code:

```
#suggest a suitable searching and sorting algorithms for Railway Ticket Reservation
#System that stores ticket ID, passenger name,train number, seat number, and travel date.and justify the choice of algorithms.
#For a Railway Ticket Reservation System, I would suggest using the Merge Sort algorithm for sorting the ticket records and the Binary search for searching through the records.
#Justification for Merge Sort:
"""1. Merge Sort is a stable sorting algorithm, which means it preserves the relative order of records with equal keys (e.g., travel date).
   This is important in a reservation system where the order of bookings may matter.
2. Merge Sort has a time complexity of O(n log n) in all cases, making it efficient for sorting large datasets,
   which is likely in a railway reservation system with many tickets."""
#Justification for Binary Search:
"""1. Binary Search is efficient for searching through sorted data, with a time complexity of O(log n).
   This makes it suitable for quickly finding specific ticket records based on criteria such as ticket ID or passenger name.
2. Since the ticket records will be sorted using Merge Sort, Binary Search can be effectively utilized to retrieve information without having to scan through the entire list, thus improving performance in a real-time reservation system.
Overall
, the combination of Merge Sort for sorting and Binary Search for searching provides an efficient and effective way to manage ticket records in a Railway Ticket Reservation System, ensuring that the system can handle large volumes of data while maintaining quick access to specific records when needed."""
# Example implementation of Merge Sort and Binary Search for the Railway Ticket Reservation System
"""class Ticket:
    def __init__(self, ticket_id, passenger_name, train_number, seat_number, travel_date):
        self.ticket_id = ticket_id
        self.passenger_name = passenger_name
        self.train_number = train_number
        self.seat_number = seat_number
        self.travel_date = travel_date
def merge_sort(tickets):
    if len(tickets) <= 1:
        return tickets
    mid = len(tickets) // 2
    left = merge_sort(tickets[:mid])
    right = merge_sort(tickets[mid:])
    return merge(left, right)
def merge(left, right):
    merged = []
    i, j = 0, 0
    while i < len(left) and j < len(right):
        if left[i].travel_date < right[j].travel_date:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged
def binary_search(tickets, target_id):
    low = 0
    high = len(tickets) - 1
    while low <= high:
        mid = (low + high) // 2
        if tickets[mid].ticket_id == target_id:
            return tickets[mid]
        elif tickets[mid].ticket_id < target_id:
            low = mid + 1
        else:
            high = mid - 1
    return None
# Example usage:
if __name__ == "__main__":
    tickets = [
        Ticket(1, "John Doe", "Train A", "Seat 1", "2024-07-01"),
        Ticket(2, "Jane Doe", "Train B", "Seat 2", "2024-07-02"),
        Ticket(3, "Alice Johnson", "Train A", "Seat 3", "2024-07-01"),
    ]
    sorted_tickets = merge_sort(tickets)
    target_id = 2
    result = binary_search(sorted_tickets, target_id)
    if result:
        print(f"Ticket found: {result.passenger_name} on {result.train_number} at {result.travel_date}")
    else:
        print("Ticket not found.")
        """
```

Output:

```
Ticket found: Jane Doe on Train B at 2024-07-02
```

Significance:

Merge Sort is used to efficiently arrange ticket records based on **travel date or seat number**, ensuring organized booking management. Binary Search allows quick lookup of tickets using **ticket ID**, reducing search time. These algorithms help the system handle large reservation datasets efficiently.

Task 5: Smart Hostel Room Allocation System

Prompt:

#Suggest optimized searching and sorting algorithms for a hostel room allocationsystem storing student ID, room number, floor, and allocation date. Implement Python functions to search by student ID and sort records based on room number or allocation date.

Code:

```python
#Suggest optimized searching and sorting algorithms for a hostel room allocation
#system storing student ID, room number, floor, and allocation date.
# Implement Python functions to search by student ID and sort records based on room number or allocation date.
#For a hostel room allocation system, I would suggest using the merge sort algorithm for sorting the records and a linear search for searching by student ID.
#Justification for Merge Sort:
"""1. Merge Sort is a stable sorting algorithm, which means it preserves the relative order of records with equal keys (e.g., allocation date).
   This is important in a room allocation system where the order of allocations may matter.
2. Merge Sort has a time complexity of O(n log n) in all cases, making
   it efficient for sorting large datasets, which is likely in a hostel room allocation system with many records."""
#Justification for Linear Search:
"""1. Linear Search is simple and effective for searching through unsorted data, with a time complexity of O(n).
   This makes it suitable for quickly finding specific records based on student ID, especially if the dataset is not sorted by student ID.
2. Since the records will be sorted using Merge Sort based on room number or allocation date, Linear Search can be effectively utilized to retrieve information without having to sort the data by student ID,
thus improving performance in a real-time allocation system.
Overall, the combination of Merge Sort for sorting and Linear Search for searching provides an efficient and effective
   way to manage room allocation records in a hostel room allocation system, ensuring that the system can handle large volumes of data while maintaining quick access to specific records when needed."""
# Example implementation of Merge Sort and Linear Search for the hostel room allocation system
class Allocation:
    def __init__(self, student_id, room_number, floor, allocation_date):
        self.student_id = student_id
        self.room_number = room_number
        self.floor = floor
        self.allocation_date = allocation_date
def merge_sort(allocations, key):
    return sorted(allocations, key=lambda x: getattr(x, key))
def linear_search(allocations, target_id):
    for allocation in allocations:
        if allocation.student_id == target_id:
            return allocation
    return None
# Example usage:
if __name__ == "__main__":
    allocations = [
        Allocation(1, "Room 101", "Floor 1", "2024-07-01"),
        Allocation(2, "Room 102", "Floor 1", "2024-07-02"),
        Allocation(3, "Room 201", "Floor 2", "2024-07-03")
    ]
    sorted_allocations = merge_sort(allocations, key='room_number')
    target_allocation = linear_search(allocations, target_id=2)
    if target_allocation:
        print(f"Allocation found: Student ID {target_allocation.student_id} in {target_allocation.room_number} on {target_allocation.allocation_date}")
    else:
        print("Allocation not found.")
```

Output:

```
Allocation found: Student ID 2 in Room 102 on 2024-07-02
```

Significance:

Merge Sort helps organize allocation records based on **room number or allocation date**, making hostel management easier. Linear Search allows simple retrieval of student room allocation using **student ID** when the dataset is not sorted by ID. This approach ensures efficient and manageable record handling.

Task 6: Online Movie Streaming Platform

Prompt:

Recommend suitable searching and sorting algorithms for a movie streaming platform storing movie ID, title, genre, rating, and release year. Implement Python functions to search movies by ID and sort movies based on rating or release year."

Code:

```python
#Recommend suitable searching and sorting algorithms for a movie streaming platform storing movie ID, title, genre, rating, and release year.
#  Implement Python functions to search movies by ID and sort movies based on rating or release year."
#For a movie streaming platform, I would recommend using the Timsort algorithm for sorting the movie records and a Hash Table for searching by movie ID.
#Justification for Timsort:
"""1. Timsort is a hybrid sorting algorithm that performs well on real-world data, with a time complexity of O(n log n) in the worst case and O(n) in the best case.
This makes it efficient for sorting movies based on rating or release year.
2. Timsort is stable, meaning it preserves the relative order of records with equal keys, which is important when sorting by release year to maintain the order of movies released in the same year."""
#Justification for Hash Table:
"""1. A Hash Table provides constant average time complexity O(1) for search operations, making it ideal for quickly retrieving movie records based on unique identifiers such as movie ID.
2. Hash Tables can handle collisions effectively, which is important in a movie streaming platform where multiple movies may have similar titles or release years.
This allows for efficient storage and retrieval of records without significant performance degradation.
Overall, the combination of Timsort for sorting and Hash Table for searching provides an efficient and effective way to manage movie records in a streaming platform,
ensuring that the system can handle a large number of records while maintaining quick access to specific records when needed."""
# Example implementation of Timsort and Hash Table for the movie streaming platform
class Movie:
    def __init__(self, movie_id, title, genre, rating, release_year):
        self.movie_id = movie_id
        self.title = title
        self.genre = genre
        self.rating = rating
        self.release_year = release_year
def timsort(movies, key):
    return sorted(movies, key=lambda x: getattr(x, key))
def create_hash_table(movies):
    hash_table = {}
    for movie in movies:
        hash_table[movie.movie_id] = movie
    return hash_table
def search_by_movie_id(hash_table, movie_id):
    return hash_table.get(movie_id, None)
# Example usage:
if __name__ == "__main__":
    movies = [
        Movie(1, "Inception", "Sci-Fi", 8.8, 2010),
        Movie(2, "The Matrix", "Action", 8.7, 1999),
        Movie(3, "Interstellar", "Sci-Fi", 8.6, 2014),
    ]
    sorted_by_rating = timsort(movies, key='rating')
    sorted_by_year = timsort(movies, key='release_year')
    hash_table = create_hash_table(movies)
    target_id = 2
    result = search_by_movie_id(hash_table, target_id)
    if result:
        print(f"Movie found: {result.title} ({result.release_year}) with rating {result.rating}")
    else:
        print("Movie not found.")
```

Output:

```
Movie found: The Matrix (1999) with rating 8.7
```

Significance:

The **Timsort algorithm (Python's built-in sorting)** efficiently sorts movies based on rating or release year while maintaining stability. A **Hash Table** enables constant-time lookup of movies using movie ID. This combination improves performance and enhances the user experience when searching and organizing movies.

Task 7: Smart Agriculture Crop Monitoring System

Prompt:

Suggest appropriate searching and sorting algorithms for a crop monitoring system storing crop ID, crop name, soil moisture level, temperature, and yield estimate. Implement Python functions to search crops by ID and sort crops based on moisture level or yield estimate.

Code:

```
#Suggest appropriate searching and sorting algorithms for a crop monitoring system storing crop ID, crop name, soil moisture level, temperature, and yield estimate.
#Implement Python functions to search crops by ID and sort crops based on moisture level or yield estimate.
#For a crop monitoring system, I would suggest using the Timsort algorithm for sorting the crop records and a Hash Table for searching by crop ID.
#Justification for Timsort:
"""1. Timsort is a hybrid sorting algorithm that performs well on real-world data, with a time complexity of O(n log n) in the worst case and O(n) in the best case.
This makes it efficient for sorting crops based on soil moisture level or yield estimate.
2. Timsort is stable, meaning it preserves the relative order of records with equal keys, which is important when sorting by yield estimate to maintain the order of crops with the same yield."""
#Justification for Hash Table:
"""1. A Hash Table provides constant average time complexity O(1) for search operations, making it ideal for quickly retrieving crop records based on unique identifiers such as crop ID.
2. Hash Tables can handle collisions effectively, which is important in a crop monitoring system where multiple crops may have similar names or moisture levels.
This allows for efficient storage and retrieval of records without significant performance degradation.
"""Overall, the combination of Timsort for sorting and Hash Table for searching provides an efficient and effective way to manage crop records in a monitoring system,
# Example implementation of Timsort and Hash Table for the crop monitoring system
"""class Crop:
    def __init__(self, crop_id, crop_name, soil_moisture_level, temperature, yield_estimate):
        self.crop_id = crop_id
        self.crop_name = crop_name
        self.soil_moisture_level = soil_moisture_level
        self.temperature = temperature
        self.yield_estimate = yield_estimate
def timsort(crops, key):
    return sorted(crops, key=lambda x: getattr(x, key))
def create_hash_table(crops):
    hash_table = {}
    for crop in crops:
        hash_table[crop.crop_id] = crop
    return hash_table
def search_by_crop_id(hash_table, crop_id):
    return hash_table.get(crop_id, None)
# Example usage:
if __name__ == "__main__":
    crops = [
        Crop(1, "Wheat", 30, 25, 1000),
        Crop(2, "Corn", 40, 30, 1500),
        Crop(3, "Rice", 35, 28, 1200),
    ]
    sorted_by_moisture = timsort(crops, key='soil_moisture_level')
    sorted_by_yield = timsort(crops, key='yield_estimate')
    hash_table = create_hash_table(crops)
    target_id = 2
    result = search_by_crop_id(hash_table, target_id)
    if result:
        print(f"Crop found: {result.crop_name} with soil moisture level {result.soil_moisture_level} and yield estimate {result.yield_estimate}")
    else:
        print("Crop not found.")
        """
```

Output:

```
Crop found: Corn with soil moisture level 40 and yield estimate 1500
```

Significance:

Timsort efficiently sorts crop data based on **soil moisture level or yield estimate**, helping farmers analyze agricultural conditions effectively. The use of a **Hash Table** allows fast retrieval of crop information using crop ID. This improves monitoring efficiency and supports better farming decisions.

Task 8: Airport Flight Management System

Prompt:

Recommend suitable searching and sorting algorithms for an airport flight management system storing flight ID, airline name, departure time, arrival time, and status. Implement Python functions to search flights by ID and sort flights by departure or arrival time.

Code:

```python
#Recommend suitable searching and sorting algorithms for an airport flight management system storing flight ID, airline name, departure time, arrival time, and status.
#Implement Python functions to search flights by ID and sort flights by departure or arrival time.
#For an airport flight management system, I would recommend using the Timsort algorithm for sorting the flight records and a Hash Table for searching by flight ID.
#Justification for Timsort:
"""1. Timsort is a hybrid sorting algorithm that performs well on real-world data, with a time complexity of O(n log n) in the worst case and O(n) in the best case. This makes it efficient for sorting flights based on departure or arrival time.
2. Timsort is stable, meaning it preserves the relative order of records with equal keys, which is important when sorting by departure time to maintain the order of flights departing at the same time."""
#Justification for Hash Table:
"""1. A Hash Table provides constant average time complexity O(1) for search operations, making it ideal for quickly retrieving flight records based on unique identifiers such as flight ID.
2. Hash Tables can handle collisions effectively, which is important in a flight management system where multiple flights may have similar departure or arrival times. This allows for efficient storage and retrieval of records without
significant performance degradation.
Overall, the combination of Timsort for sorting and Hash Table for searching provides an efficient and effective way to manage flight records in an airport flight management system,
ensuring that the system can handle a large number of records while maintaining quick access to specific records when needed."""
# Example implementation of Timsort and Hash Table for the airport flight management system
class Flight:
    def __init__(self, flight_id, airline_name, departure_time, arrival_time, status):
        self.flight_id = flight_id
        self.airline_name = airline_name
        self.departure_time = departure_time
        self.arrival_time = arrival_time
        self.status = status
def timsort(flights, key):
    return sorted(flights, key=lambda x: getattr(x, key))
def create_hash_table(flights):
    hash_table = {}
    for flight in flights:
        hash_table[flight.flight_id] = flight
    return hash_table
def search_by_flight_id(hash_table, flight_id):
    return hash_table.get(flight_id, None)
# Example usage:
if __name__ == "__main__":
    flights = [
        Flight(1, "Airline A", "2024-07-01 10:00", "2024-07-01 12:00", "On Time"),
        Flight(2, "Airline B", "2024-07-01 11:00", "2024-07-01 13:00", "Delayed"),
        Flight(3, "Airline C", "2024-07-01 09:00", "2024-07-01 11:00", "On Time"),
    ]
    sorted_by_departure = timsort(flights, key='departure_time')
    sorted_by_arrival = timsort(flights, key='arrival_time')
    hash_table = create_hash_table(flights)
    target_id = 2
    result = search_by_flight_id(hash_table, target_id)
    if result:
        print(f"Flight found: {result.airline_name} departing at {result.departure_time} with status {result.status}")
    else:
        print("Flight not found.")
```

Output:

```
Flight found: Airline B departing at 2024-07-01 11:00 with status Delayed
```

Significance:

Timsort efficiently organizes flight records based on **departure or arrival time**, which helps manage flight schedules effectively. A **Hash Table** enables quick retrieval of flight details using flight ID. This combination improves operational efficiency and ensures fast access to flight information.